



INF1416

Segurança da Informação

Prof. Anderson Oliveira da Silva
D. Sc. Ciências em Informática
Engenheiro de Computação
anderson@inf.puc-rio.br

Departamento de Informática
PUC-Rio

Java Security:

- **Arquitetura de Criptografia Java**
- **Provedores de Pacotes Criptográficos**
- **Algoritmos, Classes e Métodos**
 - Resumo de Mensagem
 - Código de Autenticação de Mensagem
 - Criptografia Simétrica e Assimétrica
 - Assinatura Digital
- **Ferramentas**
 - Keytool
 - Jarsigner
- **Suporte a SSL**

Java Security

- Segurança da Informação
Prof. Anderson O. da Silva

3



Arquitetura de Criptografia Java (Java Cryptography Architecture – JCA)

- A Security API é uma Core API de Java disponível no pacote *java.security* (e seus subpacotes).
- A primeira implementação dessa API, no JDK 1.1, introduziu a Java Cryptography Architecture (JCA), com suporte a resumo de mensagem e assinatura digital.
- O Java 2 SDK estendeu o JCA acrescentando novas funcionalidades, como suporte a certificados X.509 v3 e a *arquitetura de providers* para possibilitar implementações criptográficas múltiplas e interoperáveis.

Arquitetura de Criptografia Java (Java Cryptography Architecture – JCA)

- A Java Cryptography Extension (JCE) é uma implementação de um *Cryptography Package Provider*, ou, de forma mais simples, um *provider*, que provê um framework e implementações para cifragem, geração de chave e acordo de chave, e algoritmos para código de autenticação de mensagens.
- A JCE era um pacote (extensão) opcional no Java 2 SDK Standard Edition, versões 1.2.x e 1.3.x, e foi integrado nessa distribuição a partir da série 1.4.

Arquitetura de Criptografia Java (Java Cryptography Architecture – JCA)

- A JCE API inclui:
 - Criptografia Simétrica de Bloco, como DES, 3DES, AES, RC2, e IDEA;
 - Criptografia Simétrica de Fluxo, como RC4;
 - Criptografia Assimétrica, como RSA e EC;
 - Criptografia baseada em senha
(Password-based encryption - PBE);
 - Acordo de chaves (DH);
 - Código de Autenticação de Mensagem
(Message Authentication Codes - MAC).

Arquitetura de Criptografia Java: Pacotes de Terceiros

- A JCA pode ser estendida através de pacotes (extensões) de terceiros (provedores).
- Um exemplo de pacote é a Bouncy Castle Provider:
 - http://www.bouncycastle.org/latest_releases.html
- Pacote: bcprov-jdk15on-165.jar (para Java SDK 1.5 a 1.15)
 - Entre outros recursos, disponibiliza os clássicos algoritmos de criptografia e assinatura digital da RSA, além do TLS 1.3.
- Os pacotes devem ser registradas:
 - Estaticamente ou Dinamicamente

Registro do pacote de criptografia de terceiros:

- Quando o registro é estático, o pacote do provedor é instanciado na inicialização da máquina virtual.
- O registro dinâmico pode ser feito durante a execução do sistema através de chamadas aos métodos *addProvider* ou *insertProviderAt* da classe *Security*.

Registro estático do pacote de criptografia:

- O pacote deve ser copiado para as seguintes pastas:
 - Program Files\Java\jdk1.8.0_281\jre\lib\ext
 - Program Files\Java\jre1.8.0_281\lib\ext
- A prioridade de utilização do pacote deve ser registrada no *master security properties file* (arquivo *java.security*), localizado nas seguintes pastas:
 - Program Files\Java\jdk1.8.0_281\jre\lib\security
 - Program Files\Java\jre1.8.0_281\lib\security

Registro estático do pacote de criptografia:

- Cada provedor deve implementar uma subclasse da classe *Provider*. O nome da subclasse do provedor e sua prioridade devem ser especificadas no arquivo *java.security*, da seguinte forma:
 - `security.provider.<n>=<className>`
 - <n>, indica a preferência e 1 é a mais prioritária (reservada para a classe default);
 - <className> especifica a subclasse da classe *Provider* cujo construtor indica os valores de várias propriedades requeridas para que a Java Security API encontre os algoritmos e outras facilidades implementadas pelo *Provider*.

Registro estático do pacote de criptografia:

- Deve existir pelo menos um provedor especificado. O default provider do JDK é chamado *SUN provider*:
 - `security.provider.1=sun.security.provider.Sun`

Registro estático do pacote de criptografia:

- Por exemplo, a especificação do pacote `bcprov-jdk15on-165.jar` é feito da seguinte maneira:

```
#  
# List of providers and their preference orders (see above):  
#  
  
security.provider.1=sun.security.provider.Sun  
security.provider.2=sun.security.rsa.SunRsaSign  
security.provider.3=sun.security.ec.SunEC  
security.provider.4=com.sun.net.ssl.internal.ssl.Provider  
security.provider.5=com.sun.crypto.provider.SunJCE  
security.provider.6=sun.security.jgss.SunProvider  
security.provider.7=com.sun.security.sasl.Provider  
security.provider.8=org.jcp.xml.dsig.internal.dom.XMLDSigRI  
security.provider.9=sun.security.smartcardio.SunPCSC  
security.provider.10=sun.security.mscapi.SunMSCAPI  
security.provider.11=org.bouncycastle.jce.provider.BouncyCastleProvider
```

Determinando os providers registrados no Java Runtime:

- O JDK 1.5 e superiores disponibilizam a classe *Providers* para consulta a diversas informações sobre os providers registrados.
- O `ListProvidersExample` a seguir utiliza os seguintes métodos:
 - `Security.getProviders()`: Retorna um array de objetos `Providers`.
 - `.getName()`: Retorna o nome de um provedor.
 - `.getVersion()`: Retorna a versão de um provedor.

- ListProvidersExample

```
import java.security.Provider;
import java.security.Security;
//
// lista os providers registrados no Java Runtime
//
public class ListProvidersExample
{
    public static void main(String[] args)
    {
        Provider[] providers = Security.getProviders();
        for (int i = 0; i != providers.length; i++)
        {
            System.out.println("Name: " + providers[i].getName()
                               + "    Version: " + providers[i].getVersion());
        }
    }
}
```

Determinando os providers registrados no Java Runtime:

- **Compilação e execução do ListProvidersExample:**

```
> javac ListProvidersExample.java
```

```
> java ListProvidersExample
```

```
Name: SUN    Version: 1.8
```

```
Name: SunRsaSign    Version: 1.8
```

```
Name: SunEC    Version: 1.8
```

```
Name: SunJSSE    Version: 1.8
```

```
Name: SunJCE    Version: 1.8
```

```
Name: SunJGSS    Version: 1.8
```

```
Name: SunSASL    Version: 1.8
```

```
Name: XMLDSig    Version: 1.8
```

```
Name: SunPCSC    Version: 1.8
```

```
Name: SunMSCAPI    Version: 1.8
```

```
Name: BC    Version: 1.65
```

Algoritmos, classes e métodos:

- O JDK 1.8 suporta os seguintes algoritmos de message digest:
 - MD2 and MD5 , que geram digest de 128 bits;
 - SHA-1, que gera digest de 160 bits;
 - SHA-256 e SHA-512, que geram digest de 256 e 512 bits, respectivamente;
- A classe MessageDigest manipula com message digests.
- O MessageDigestExample a seguir utiliza os seguintes métodos:
 - MessageDigest.getInstance("SHA-256") : Cria o message digest.
 - update(plaintext) : Calcula o digest a partir de um texto plano.
 - digest() : Retorna o message digest calculado.

- MessageDigestExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// Gera o message digest do argumento passado na linha de comando
public class MessageDigestExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java MessageDigestExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // define o objeto messageDigest com o algoritmo SHA-256
        MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
        //
        // imprime o provider utilizado
        System.out.println( "\n" + messageDigest.getProvider().getInfo() );
    }
}
```

- MessageDigestExample (parte 2)**

```
//  
// calcula o digest e imprime seu tamanho em bits  
messageDigest.update( plainText);  
byte [] digest = messageDigest.digest();  
System.out.println( "\nDigest length: " + digest.length * 8 + "bits" );  
  
// converte o digest para hexadecimal  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < digest.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (digest[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o digest em hexadecimal  
System.out.println( "\nDigest(hex): " );  
System.out.println( buf.toString() );  
}  
}
```

Algoritmos, classes e métodos:

- Compilação e execução do MessageDigestExample:**

> javac MessageDigestExample.java

> java MessageDigestExample "Este eh um teste!"

SUN (DSA key/parameter generation; DSA signing; SHA-1, MD5 digests; SecureRandom; X.509 certificates; JKS & DKS keystores; PKIX CertPathValidator; PKIX CertPathBuilder; LDAP, Collection CertStores, JavaPolicy Policy; JavaLoginConfig Configuration)

Digest length: 256bits

Digest(hex):

bae8f0ffa90cb82565e8f745e21a881a1649af5432d19f0b8abc90aa652165ab

Algoritmos, classes e métodos:

- O JDK 1.8 suporta algoritmos de message authentication code, ou seja, a combinação de chave com algoritmo de message digest. Os algoritmos suportados são:
 - HMAC/SHA-256
 - HMAC/SHA-1
 - HMAC/MD5
- A classe Mac manipula com message-authentication codes utilizando chaves produzidas pela classe KeyGenerator.
- Essas classes são encontradas em um provider criptográfico default disponível no JDK.

Algoritmos, classes e métodos:

- O `MessageAuthenticationCodeExample` a seguir utiliza os seguintes métodos:
 - `KeyGenerator.getInstance("HmacSHA256")` e `.generateKey()` : Gera a chave adequada.
 - `Mac.getInstance("HmacSHA256")` : Cria o objeto Mac.
 - `.init(Mackey)` : Inicializa o objeto Mac.
 - `.update(plaintext)` and `.doFinal()` : Calcula o MAC com o texto plano.

- MessageAuthenticationCodeExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// Gera o Message Authentication Code (MAC)
public class MessageAuthenticationCodeExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java MessageAuthenticationCodeExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //

        // gera uma chave para o algoritmo Hmac
        System.out.println( "\nStart generating key" );
        KeyGenerator keyGen = KeyGenerator.getInstance("HmacSHA256");
        SecretKey Mackey = keyGen.generateKey();
        System.out.println( "Finish generating key" );
    }
}
```

- MessageAuthenticationCodeExample (parte 2)**

```
//  
// define um objeto MAC e o atualiza com o texto plano  
Mac mac = Mac.getInstance("HmacSHA256");  
mac.init(Mackey);  
mac.update(plainText);  
//  
// imprime a informacao do provider utilizado  
System.out.println( "\n" + mac.getProvider().getInfo() );  
  
// converte o MAC para hexadecimal  
byte[] macFinal = mac.doFinal();  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < macFinal.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (macFinal[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o MAC em hexadecimal  
System.out.println( "\nMAC: " );  
System.out.println( buf.toString() );  
}  
}
```

Algoritmos, classes e métodos:

- Compilação e execução do MessageAuthenticationCodeExample:**

```
> javac MessageAuthenticationCodeExample.java
```

```
> java MessageAuthenticationCodeExample "Este eh um teste!"
```

Start generating key

Finish generating key

SunJCE Provider (implements RSA, DES, Triple DES, AES, Blowfish, ARCFOUR, RC2, PBE, Diffie-Hellman, HMAC)

MAC:

8f41d228207db64b1d8166eeca22ca0acfee6b8c86863961cee849392cdd35ed

Algoritmos, classes e métodos:

- O JDK 1.8 suporta vários métodos de preenchimento (padding), modos de operação da criptografia (ex: ECB e CBC) e algoritmos de criptografia simétricos (ex: DES, RC4).
- Métodos de preenchimento:
 - No padding
 - PKCS5
 - OAEP
 - SSL3

Algoritmos, classes e métodos:

- **Modos de operação da criptografia:**
 - ECB (Electronic Code Book)
 - CBC (Cipher Block Chaining)
 - CFB (Cipher Feedback Mode)
 - OFB (Output Feedback Mode)
- **Algoritmos de criptografia:**
 - DES (Data Encryption Standard) e TripleDES
 - AES
 - RC2, RC4, and RC5
 - Blowfish
 - PBE (Password Based Encryption)

Algoritmos, classes e métodos:

- A classe Cipher manipula com algoritmos de criptografia simétricos com chaves produzidas pela classe KeyGenerator.
- O SymmetricKeyCipherECBExample a seguir utiliza os seguintes métodos:
 - KeyGenerator.getInstance("DES") , .init(56) , e .generateKey(): Gera a chave.
 - Cipher.getInstance("DES/ECB/PKCS5Padding"): Cria o objeto Cipher (especifica o algoritmo, modo e preenchimento).
 - .init(Cipher.ENCRYPT_MODE, key): Inicializa o objeto Cipher.
 - .doFinal(plainText): Calcula o texto cifrado a partir do texto plano.
 - .init(Cipher.DECRYPT_MODE, key): Decifra o texto cifrado.
 - .doFinal(cipherText): Computa o texto cifrado.

- SymmetricKeyCipherECBExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// criptografa e descriptografa utilizando DES
public class SymmetricKeyCipherECBExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java SymmetricKeyCipherExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // gera uma chave para o DES
        System.out.println( "\nStart generating DES key" );
        KeyGenerator keyGen = KeyGenerator.getInstance("DES");
        keyGen.init(56);
        Key key = keyGen.generateKey();
        System.out.println( "Finish generating DES key" );
    }
}
```

- SymmetricKeyCipherECBExample (parte 2)**

```
//  
// define um objeto de cifra DES e imprime o provider utilizado  
Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");  
System.out.println( "\n" + cipher.getProvider().getInfo() );  
//  
// criptografa utilizando a chave e o texto plano  
System.out.println( "\nStart encryption" );  
cipher.init(Cipher.ENCRYPT_MODE, key);  
byte[] cipherText = cipher.doFinal(plainText);  
System.out.println( "Finish encryption (hex output): " );  
  
// converte o cipherText para hexadecimal  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < cipherText.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (cipherText[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o ciphertext em hexadecimal  
System.out.println( buf.toString() );
```

- SymmetricKeyCipherECBExample (parte 3)**

```
//  
// descriptografa o texto cifrado com a chave  
System.out.println( "\nStart decryption" );  
cipher.init(Cipher.DECRYPT_MODE, key);  
byte[] newPlainText = cipher.doFinal(cipherText);  
System.out.println( "Finish decryption: " );  
System.out.println( new String(newPlainText, "UTF8") );  
}  
}
```

Algoritmos, classes e métodos:

- **Compilação e execução do SymmetricKeyCipherECBExample:**

```
> javac SymmetricKeyCipherECBExample.java
```

```
> java SymmetricKeyCipherECBExample "Este eh um teste!"
```

Start generating DES key

Finish generating DES key

SunJCE Provider (implements RSA, DES, Triple DES, AES, Blowfish, ARCFOUR, RC2, PBE, Diffie-Hellman, HMAC)

Start encryption

Finish encryption (hex output):

ad944b0707c1f7d8c32db677313140b9e1cdff0f0671be4c

Start decryption

Finish decryption:

Este eh um teste!

Algoritmos, classes e métodos:

- A classe Cipher manipula com algoritmos de criptografia simétricos com chaves produzidas pela classe KeyGenerator.
- O SymmetricKeyCipherCBCExample a seguir utiliza os seguintes métodos:
 - KeyGenerator.getInstance("DES") , .init(56) , e .generateKey(): Gera a chave.
 - IvParameterSpec(iv): Cria um IV a partir de um array de 8 bytes.
 - Cipher.getInstance("DES/CBC/PKCS5Padding"): Cria o objeto Cipher (especifica o algoritmo, modo e preenchimento).
 - .init(Cipher.ENCRYPT_MODE, key, ivSpec): Inicializa o objeto Cipher.
 - .doFinal(plainText): Calcula o texto cifrado a partir do texto plano.
 - .init(Cipher.DECRYPT_MODE, key, ivSpec): Decifra o texto cifrado.
 - .doFinal(cipherText): Computa o texto cifrado.

- SymmetricKeyCipherCBCExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// criptografa e descriptografa utilizando DES
public class SymmetricKeyCipherCBCExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java SymmetricKeyCipherExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // gera uma chave para o DES
        System.out.println( "\nStart generating DES key" );
        KeyGenerator keyGen = KeyGenerator.getInstance("DES");
        keyGen.init(56);
        Key key = keyGen.generateKey();
        System.out.println( "Finish generating DES key" );
    }
}
```

- SymmetricKeyCipherCBCExample (parte 2)**

```
//  
// define um IV para o DES (8 bytes)  
System.out.println( "\nStart creating DES IV" );  
byte[] iv = new byte[] { 0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7 };  
IvParameterSpec ivSpec = new IvParameterSpec(iv);  
System.out.println( "Finish creating DES IV" );  
//  
// define um objeto de cifra DES e imprime o provider utilizado  
Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");  
System.out.println( "\n" + cipher.getProvider().getInfo() );  
//  
// criptografa utilizando a chave e o texto plano  
System.out.println( "\nStart encryption" );  
cipher.init(Cipher.ENCRYPT_MODE, key, ivSpec);  
byte[] cipherText = cipher.doFinal(plainText);  
System.out.println( "Finish encryption (hex output): " );
```

- SymmetricKeyCipherCBCExample (parte 3)**

```
// converte o cipherText para hexadecimal
StringBuffer buf = new StringBuffer();
for(int i = 0; i < cipherText.length; i++) {
    String hex = Integer.toHexString(0x0100 + (cipherText[i] & 0x00FF)).substring(1);
    buf.append((hex.length() < 2 ? "0" : "") + hex);
}

// imprime o ciphertext em hexadecimal
System.out.println( buf.toString() );//
// descriptografa o texto cifrado com a chave
System.out.println( "\nStart decryption" );
cipher.init(Cipher.DECRYPT_MODE, key, ivSpec);
byte[] newPlainText = cipher.doFinal(cipherText);
System.out.println( "Finish decryption: " );
System.out.println( new String(newPlainText, "UTF8") );
}
```

Algoritmos, classes e métodos:

- Compilação e execução do SymmetricKeyCipherCBCExample:**

```
> javac SymmetricKeyCipherCBCExample.java
```

```
> java SymmetricKeyCipherCBCExample "Este eh um teste!"
```

Start generating DES key

Finish generating DES key

Start creating DES IV

Finish creating DES IV

SunJCE Provider (implements RSA, DES, Triple DES, AES, Blowfish, ARCFOUR, RC2, PBE, Diffie-Hellman, HMAC)

Start encryption

Finish encryption (hex output):

cdc6cbc6ba6951bf3c89d05e9986e6acc981d08c4e4251b4

Start decryption

Finish decryption:

Este eh um teste!

Algoritmos, classes e métodos:

- O JDK 1.8 suporta algoritmos de criptografia assimétricos, como:
 - RSA
 - Diffie-Hellman
- A classe Cipher manipula com algoritmos de criptografia assimétricos com chaves produzidas pela classe KeyPairGenerator.

Algoritmos, classes e métodos:

- O `AsymmetricKeyCipherSunJCEExample` a seguir utiliza os seguintes métodos:
 - `KeyPairGenerator.getInstance("RSA")` , `.initialize(2048)` , e `.generateKeyPair()` : Gera o par de chaves.
 - `Cipher.getInstance("RSA/ECB/PKCS1Padding")`: Cria o objeto Cipher (especifica o algoritmo, modo, padding).
 - `.init(Cipher.ENCRYPT_MODE, key.getPublic())` : Inicializa o objeto Cipher.
 - `.doFinal(plainText)` : Calcula o texto cifrado a partir do texto plano.
 - `.init(Cipher.DECRYPT_MODE, key.getPrivate())` e `.doFinal(cipherText)` : Decifra o texto cifrado.

- AsymmetricKeyCipherSunJCEExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// criptografia assimetrica utilizando o RSA.
public class AsymmetricKeyCipherSunJCEExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java AsymmetricKeyCipherExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // gera um par de chaves RSA
        System.out.println( "\nStart generating RSA key" );
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048);
        KeyPair key = keyGen.generateKeyPair();
        System.out.println( "Finish generating RSA key" );
    }
}
```

- AsymmetricKeyCipherSunJCEExample (parte 2)**

```
//  
// define o objeto de cifra RSA e imprime o provider utilizado  
Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");  
System.out.println( "\n" + cipher.getProvider().getInfo() );  
//  
// criptografa o texto plano utilizando a chave publica  
System.out.println( "\nStart encryption" );  
cipher.init(Cipher.ENCRYPT_MODE, key.getPublic());  
byte[] cipherText = cipher.doFinal(plainText);  
System.out.println( "Finish encryption (hex output): " );  
  
// converte o cipherText para hexadecimal  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < cipherText.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (cipherText[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o ciphertext em hexadecimal  
System.out.println( buf.toString() );
```

- AsymmetricKeyCipherSunJCEExample (parte 3)**

```
//  
// descriptografa o texto cifrado utilizando a chave privada  
System.out.println( "\nStart decryption" );  
cipher.init(Cipher.DECRYPT_MODE, key.getPrivate());  
byte[] newPlainText = cipher.doFinal(cipherText);  
System.out.println( "Finish decryption: " );  
System.out.println( new String(newPlainText, "UTF8") );  
}  
}
```

Algoritmos, classes e métodos:

- Compilação e execução do AsymmetricKeyCipherSunJCEExample:**

> javac AsymmetricKeyCipherSunJCEExample.java

> java AsymmetricKeyCipherSunJCEExample "Este eh um teste!"

Start generating RSA key

Finish generating RSA key

SunJCE Provider (implements RSA, DES, Triple DES, AES, Blowfish, ARCFOUR, RC2, PBE, Diffie-Hellman, HMAC)

Start encryption

Finish encryption (hex output):

```
788b9cae3a7ea18067d1a246599e11a14ff8ad5276da4e7772eeea77f718cafc3f3015a4cf2daee150984beb40e38b347f
3f6a4c3d80dda83ea27f6d24fb746fa0e463ad7f62f8c557ba91996ce5e7ca3dae1d428f1c4bbf7cdb7086aa1a6c143ae80
0cd79e9aa9c79e905ec565f94310c60b650e872991a34d90c30ee6fbf53086e54236923c99e35b6e13404a87ee557fec5
9bb5707361e8654d21cad181afeafe63ac7c652a5e835f2ee06ab3371df31f7977d3b7b66d578ad39b9241304a846ea0f
481e35ecd260f2af4a264a659b2869d6888ba3c027a30044d70914dbe62af6ff3ce2501bb6fa2f95d97a58f48cc034e91e9
78366b40d40563b3366a2a
```

Start decryption

Finish decryption:

Este eh um teste!

Algoritmos, classes e métodos:

- O `AsymmetricKeyCipherBCExample` a seguir utiliza os seguintes métodos:
 - `addProvider(p)`: Carrega o provider criptográfico (p).
 - `KeyPairGenerator.getInstance("RSA")` , `.initialize(2048)` , e `.generateKeyPair()` : Gera o par de chaves.
 - `Cipher.getInstance("RSA/ECB/PKCS1Padding", "BC")`: Cria o objeto Cipher (especifica o algoritmo, modo, padding e provider BouncyCastle).
 - `.init(Cipher.ENCRYPT_MODE, key.getPublic())` : Inicializa o objeto Cipher.
 - `.doFinal(plainText)` : Calcula o texto cifrado a partir do texto plano.
 - `.init(Cipher.DECRYPT_MODE, key.getPrivate())` e `.doFinal(cipherText)` : Decifra o texto cifrado.

- AsymmetricKeyCipherBCExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// criptografia assimetrica utilizando o RSA.
public class AsymmetricKeyCipherSunJCEExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java AsymmetricKeyCipherExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");

        //
        // Carrega o provider BouncyCastle
        Provider p = new org.bouncycastle.jce.provider.BouncyCastleProvider();
        int result = Security.addProvider(p);
```

- AsymmetricKeyCipherBCExample (parte 2)

```
//  
// gera um par de chaves RSA  
System.out.println( "\nStart generating RSA key" );  
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");  
keyGen.initialize(2048);  
KeyPair key = keyGen.generateKeyPair();  
System.out.println( "Finish generating RSA key" );  
  
//  
// define o objeto de cifra RSA e imprime o provider utilizado  
Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding", "BC");  
System.out.println( "\n" + cipher.getProvider().getInfo() );  
//  
// criptografa o texto plano utilizando a chave publica  
System.out.println( "\nStart encryption" );  
cipher.init(Cipher.ENCRYPT_MODE, key.getPublic());  
byte[] cipherText = cipher.doFinal(plainText);  
System.out.println( "Finish encryption (hex output): " );
```

- AsymmetricKeyCipherBCExample (parte 3)**

```
// converte o cipherText para hexadecimal
StringBuffer buf = new StringBuffer();
for(int i = 0; i < cipherText.length; i++) {
    String hex = Integer.toHexString(0x0100 + (cipherText[i] & 0x00FF)).substring(1);
    buf.append((hex.length() < 2 ? "0" : "") + hex);
}

// imprime o ciphertext em hexadecimal
System.out.println( buf.toString() );

//
// descriptografa o texto cifrado utilizando a chave privada
System.out.println( "\nStart decryption" );
cipher.init(Cipher.DECRYPT_MODE, key.getPrivate());
byte[] newPlainText = cipher.doFinal(cipherText);
System.out.println( "Finish decryption: " );
System.out.println( new String(newPlainText, "UTF8") );
}
}
```

Algoritmos, classes e métodos:

- Compilação e execução do AsymmetricKeyCipherBCExample:**

```
> javac -classpath ./bcprov-jdk15on-165.jar AsymmetricKeyCipherBCExample.java
```

```
> java -classpath ./bcprov-jdk15on-165.jar AsymmetricKeyCipherBCExample "Este eh um teste!"
```

Start generating RSA key

Finish generating RSA key

BouncyCastle Security Provider v1.65

Start encryption

Finish encryption (hex output):

```
0b95639289e1fd29c09fc986e4279a6b795b151aa2d0d7537914108e1611a58f4cb6222ece195c999c7bf295c0d8cdead  
683fe85b57fb7baa397b61341274a792bce979f618a3243a241e4800b2df15b8e14b81f11a47756a6bf53f4a6c48c936ef4  
2371b5623a564164bdf8e2fa1399dbc16bd04498d59b328a64311e1427b299149d40e2127c5f8f7266925eb5536ca7ef3  
060c51c68bbb8bf52c5ddd2f35753d8d3e7603dce16470f0df48c567875d500dd99a856e263020273417d34cb1ab251ca  
d3ceaa12fc65d37b5c1653238b2bd8617a7a53adb0443fa1f89923bc8043cf87c3e5a2dee473420d714f0c4e6be7ae9dc  
dd698c57e735c942ab8b759c9
```

Start decryption

Finish decryption:

Este eh um teste!

Algoritmos, classes e métodos:

- O JDK 1.8 suporta os seguintes algoritmos de assinatura digital:
 - MD2/RSA e MD5/RSA
 - SHA1/DSA
 - SHA1/RSA, SHA256/RSA, SHA384/RSA e SH512/RSA
 - SHA1/ECDSA, SHA256/ECDSA, SHA384/ECDSA e SH512/ECDSA
- Existem, basicamente, duas formas de trabalhar com assinaturas digitais:
 - Combinando os procedimentos anteriores;
 - Utilizando diretamente os recursos da linguagem Java.
 - Utilizado no DigitalSignatureExample e DigitalSignatureExample2.

Algoritmos, classes e métodos:

- A classe `Signature` manipula assinaturas digitais com chaves produzidas pela classe `KeyPairGenerator`.
- O `DigitalSignatureExample` a seguir utiliza os seguintes métodos:
 - `KeyPairGenerator.getInstance("RSA")` , `.initialize(2048)` , e `.generateKeyPair()` : Gera as chaves.
 - `Signature.getInstance("SHA1WithRSA")` : Cria o objeto `Signature`.
 - `.initSign(key.getPrivate())` : Inicializa o objeto `Signature`.
 - `.update(plainText)` e `.sign()` : Calcula a assinatura a partir do texto plano.
 - `.initVerify(key.getPublic())` e `.verify(signature)` : Verifica a assinatura.

- DigitalSignatureExample (parte 1)**

```
import java.security.*;
import javax.crypto.*;
//
// este exemplo utiliza facilidades para a geracao e verificacao de assinatura digital
public class DigitalSignatureExample {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java DigitalSignatureExample text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // gera o par de chaves RSA
        System.out.println( "\nStart generating RSA key" );
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048);
        KeyPair key = keyGen.generateKeyPair();
        System.out.println( "Finish generating RSA key" );
    }
}
```

- DigitalSignatureExample (parte 2)**

```
//  
// define um objeto signature para utilizar SHA1 e RSA  
// e assina o texto plano com a chave privada,  
// o provider utilizado tambem eh impresso  
Signature sig = Signature.getInstance("SHA1WithRSA");  
sig.initSign(key.getPrivate());  
sig.update(plainText);  
byte[] signature = sig.sign();  
System.out.println( sig.getProvider().getInfo() );  
System.out.println( "\nSignature:" );  
  
// converte o signature para hexadecimal  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < signature.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (signature[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o signature em hexadecimal  
System.out.println( buf.toString() );
```

- DigitalSignatureExample (parte 3)**

```
//  
// verifica a assinatura com a chave publica  
System.out.println( "\nStart signature verification" );  
sig.initVerify(key.getPublic());  
sig.update(plainText);  
try {  
    if (sig.verify(signature)) {  
        System.out.println( "Signature verified" );  
    } else System.out.println( "Signature failed" );  
} catch (SignatureException se) {  
    System.out.println( "Singature failed" );  
}  
}  
}
```

Algoritmos, classes e métodos:

- Compilação e execução do DigitalSignatureExample:

```
> javac DigitalSignatureExample.java
```

```
> java DigitalSignatureExample "Este eh um teste!"
```

Start generating RSA key

Finish generating RSA key

Sun RSA signature provider

Signature:

```
0146e2cd51eaa7a3794e5b423431ce7cfca3716c05799d859ee7266fcf3aaaa5d1939adf3bfc4ba301ad85f85c20a6f5ac  
bfd494ad49bb27ab1a541045aeceebfd462563b7630041fe64cb0c27aa5dd5a95c0f507c34d348385d333392ccc9cc5b  
c9eaaed2e21850b2e47d7373f643fd6b5ce89c52442e468d4b22701d663f490586dc23b874e4c72a54c0e02dcbc3ead3  
ea5a51d2a56e7cecb07c71de794b23217085627ee1cc38e6e5215f06c66c951e2c1c280ac802a07bef9e30a921de185  
338655ddf3cbba32d6a2c617fbec7c9c4c15a7b5216b39e501038b0165c7d1807a0d1094cb331fa300c0409d24ea0d58  
b330a909c36a171b003ce2d7f29825a
```

Start signature verification

Signature verified

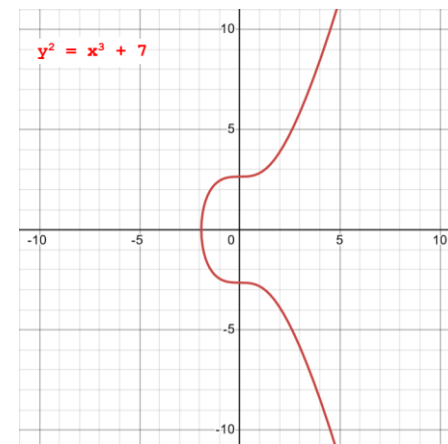
Algoritmos, classes e métodos:

- A classe `Signature` manipula assinaturas digitais com chaves produzidas pela classe `KeyPairGenerator` em conjunto com a classe `ECGenParameterSpec`.
- O `DigitalSignatureExample` a seguir utiliza os seguintes métodos:
 - `KeyPairGenerator.getInstance("EC")` , `ECGenParameterSpec("secp256k1")`, `.initialize(keyParams)` , e `.generateKeyPair()` : Gera as chaves.
 - `Signature.getInstance("SHA256WithECDSA")` : Cria o objeto `Signature`.
 - `.initSign(key.getPrivate())` : Inicializa o objeto `Signature`.
 - `.update(plainText)` e `.sign()` : Calcula a assinatura a partir do texto plano.
 - `.initVerify(key.getPublic())` e `.verify(signature)` : Verifica a assinatura.

Algoritmos, classes e métodos:

- SECG – Elliptic Curve Domain Parameters – secp256k1**

- Corpo finito $F_p = \{0, \dots, p-1\}$, onde p é um número primo:
 - $p = \text{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFC2F}$
 - $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1 = 2^{256} - 2^{32} - 977$
- Curva Elíptica:
 - $y^2 = x^3 + ax + b$, onde $a = 0$ e $b = 7$, ou seja, $y^2 = x^3 + 7$.
- Ponto Base $G (x_G, y_G)$, também conhecido como Gerador:
 - Forma comprimida:
 - $G = \text{02 79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798} = x_G \text{ e } y_G = (x_G + 7)^{1/2}$
 - Forma descomprimida:
 - $G = \text{04 79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798 483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8}$
- Ordem n de G (valor máximo da chave privada delimitado por $[1, n-1]$) e o cofator (h):
 - $n = \text{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF BAAEDCE6 AF48A03B BFD25E8C D0364141}$
 - $h = 01$



- DigitalSignatureExample2 (parte 1)**

```
import java.security.*;
import javax.crypto.*;
import java.security.spec.*;
//
// este exemplo utiliza facilidades para a geracao e verificacao de assinatura digital
public class DigitalSignatureExample2 {
    public static void main (String[] args) throws Exception {
        //
        // verifica args e recebe o texto plano
        if (args.length !=1) {
            System.err.println("Usage: java DigitalSignatureExample2 text");
            System.exit(1);
        }
        byte[] plainText = args[0].getBytes("UTF8");
        //
        // gera o par de chaves EC
        System.out.println( "\nStart generating EC key" );
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("EC");
        ECGenParameterSpec keyParams = new ECGenParameterSpec("secp256k1");
        keyGen.initialize(keyParams);
        KeyPair key = keyGen.generateKeyPair();
        System.out.println( "Finish generating EC key" );
    }
}
```

- DigitalSignatureExample2 (parte 2)**

```
//  
// define um objeto signature para utilizar SHA256 e ECDSA  
// e assina o texto plano com a chave privada,  
// o provider utilizado tambem eh impresso  
Signature sig = Signature.getInstance("SHA256WithECDSA");  
sig.initSign(key.getPrivate());  
sig.update(plainText);  
byte[] signature = sig.sign();  
System.out.println( sig.getProvider().getInfo() );  
System.out.println( "\nSignature:" );  
  
// converte o signature para hexadecimal  
StringBuffer buf = new StringBuffer();  
for(int i = 0; i < signature.length; i++) {  
    String hex = Integer.toHexString(0x0100 + (signature[i] & 0x00FF)).substring(1);  
    buf.append((hex.length() < 2 ? "0" : "") + hex);  
}  
  
// imprime o signature em hexadecimal  
System.out.println( buf.toString() );
```

- DigitalSignatureExample2 (parte 3)**

```
//  
// verifica a assinatura com a chave publica  
System.out.println( "\nStart signature verification" );  
sig.initVerify(key.getPublic());  
sig.update(plainText);  
try {  
    if (sig.verify(signature)) {  
        System.out.println( "Signature verified" );  
    } else System.out.println( "Signature failed" );  
} catch (SignatureException se) {  
    System.out.println( "Singature failed" );  
}  
}  
}
```

Algoritmos, classes e métodos:

- **Compilação e execução do DigitalSignatureExample2:**

```
> javac DigitalSignatureExample2.java
```

```
> java DigitalSignatureExample2 "Este eh um teste!"
```

Start generating EC key

Finish generating EC key

Sun Elliptic Curve provider (EC, ECDSA, ECDH)

Signature:

```
304402201cde0dfa6a5f422b4570e33055d94f68134ae6bbe780670e4088d4be1309d8cd0220755ede63ea12cd82c47  
44e641c992b5956217f4e7e7ac7be4ca74c0fcec6b654a
```

Start signature verification

Signature verified

Ferramenta para manipulação de chaves assimétricas e certificados:

- A plataforma Java utiliza um repositório para chaves e certificados conhecido como *keystore*. Esse repositório é fisicamente representado por um arquivo cujo nome default é *.keystore*. Opcionalmente, esse arquivo pode ser criptografado.
- Podem ser atribuídos nomes (aliases) às chaves e certificados para facilitar sua manipulação.
- O keystore é protegido por uma senha e a mesma pode ser utilizada para proteger os alias criados.

Ferramenta para manipulação de chaves assimétricas e certificados:

- A ferramenta **keytool** é utilizada para manipular o keystore. Existem várias opções disponíveis.
- As opções básicas para geração do par de chaves assimétricas e o correspondente certificado digital são apresentadas no exemplo a seguir. Também é apresentada a opção para visualizar o conteúdo do keystore criado.
- A ferramenta também pode ser utilizada para exportar uma chave para um arquivo no formato X.509 (CSR) para ser assinado por uma CA e então re-importado para o keystore.

- Exemplo de utilização do keytool para gerar um par de chaves assimétricas e um certificado auto-assinado: (parte1)**

```
> keytool -genkey -v -alias AndersonKey -keyalg RSA -storetype pkcs12
```

```
Enter keystore password: teste123
```

```
Re-enter new password: teste123
```

```
What is your first and last name?
```

```
[Unknown]: Anderson da Silva
```

```
What is the name of your organizational unit?
```

```
[Unknown]: DI
```

```
What is the name of your organization?
```

```
[Unknown]: PUC-Rio
```

```
What is the name of your City or Locality?
```

```
[Unknown]: Rio de Janeiro
```

```
What is the name of your State or Province?
```

```
[Unknown]: Rio de Janeiro
```

```
What is the two-letter country code for this unit?
```

```
[Unknown]: BR
```

```
Is CN=Anderson da Silva, OU=CCE, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro,
```

```
C=BR correct?
```

```
[no]: yes
```

- Exemplo de utilização do keytool para gerar um par de chaves assimétricas e um certificado auto-assinado: (parte2)

```
> keytool -genkey -v -alias AndersonKey -keyalg RSA -storetype pkcs12
```

.....

Generating 2.048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 90 days

for: CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR

Enter key password for <AndersonKey>

(RETURN if same as keystore password):

[Storing C:\Users\lander\keystore]

- Exemplo de utilização do keytool para converter o keystore do padrão proprietário JKS keystore para o padrão PKCS12:**

```
> keytool -importkeystore -srckeystore C:\Users\lander\.keystore -destkeystore C:\Users\lander\.keystore  
-deststoretype pkcs12
```

Enter source keystore password: teste123

Entry for alias andersonkey successfully imported.

Import command completed: 1 entries successfully imported, 0 entries failed or cancelled

Warning:

Migrated "C:\Users\lander\.keystore" to Non JKS/JCEKS. The JKS keystore is backed up as "C:\Users\lander\.keystore.old".

- Exemplo de utilização do keytool para consultar os dados de AndersonKey no keystore criado:

```
> keytool -list -v -alias AndersonKey
```

```
Enter keystore password:
```

```
Alias name: AndersonKey
```

```
Creation date: 12/04/2020
```

```
Entry type: PrivateKeyEntry
```

```
Certificate chain length: 1
```

```
Certificate[1]:
```

```
Owner: CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR
```

```
Issuer: CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR
```

```
Serial number: 64faf8fd
```

```
Valid from: Sun Apr 12 22:05:58 BRT 2020 until: Sat Jul 11 22:05:58 BRT 2020
```

```
Certificate fingerprints:
```

```
MD5: 6E:AF:6D:13:99:B0:E7:13:5E:59:A1:43:D4:98:85:3C
```

```
SHA1: 94:09:BB:2F:DD:49:6C:A6:AE:41:66:C4:D9:24:69:AD:78:52:C8:EF
```

```
SHA256: E7:1D:1F:02:37:22:32:C2:BB:56:0D:A4:2E:EC:1B:40:62:A9:35:13:1A:4B:43:C1:93:5E:5F:FB:43:72:DE:6A
```

```
Signature algorithm name: SHA256withRSA
```

```
Subject Public Key Algorithm: 2048-bit RSA key
```

```
Version: 3
```

Ferramenta para assinatura de código (arquivos JAR):

- A ferramenta jarsigner recebe como entrada um arquivo JAR, uma chave privada e o respectivo certificado, e gera uma versão assinada do arquivo JAR como saída.
- O message digest de cada class presente no arquivo JAR é calculado para, em seguida, ser criptografado com a chave privada.
- Essa operação garante a integridade do arquivo JAR e a identificação do dono desse arquivo.

- Criando a ListProvidersAppletExample:**

```
import java.applet.Applet;
import java.awt.Graphics;
import java.security.Provider;
import java.security.Security;

public class ListProvidersAppletExample extends Applet {
    public void paint(Graphics g) {
        Provider[] providers = Security.getProviders();
        for (int i = 0; i != providers.length; i++)
        {
            g.drawString("Name: " + providers[i].getName()
                + "    Version: " + providers[i].getVersion(),
                50, 25 * (i+1));
        }
    }
}
```

- Criando o arquivo ListProvidersAppletExample.html para apresentação da ListProvidersAppletExample.class:

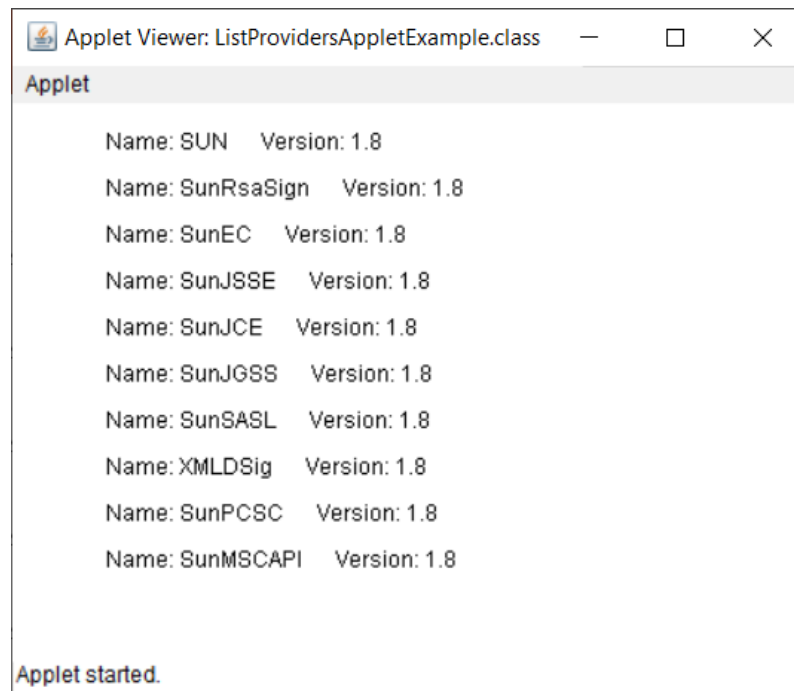
```
<HTML>
<HEAD>
<TITLE> List Providers Applet Example Program </TITLE>
</HEAD>
<BODY>

<APPLET CODE="ListProvidersAppletExample.class" WIDTH=300 HEIGHT=350>
</APPLET>
</BODY>
</HTML>
```

- Compilando e executando a ListProvidersAppletExample:

> javac ListProvidersAppletExample.java

> appletviewer ListProvidersAppletExample.html



- Criando e assinando o ListProvidersAppletExample.jar:**

```
> jar cvf ListProvidersAppletExample.jar ListProvidersAppletExample.class
```

```
added manifest
```

```
adding: ListProvidersAppletExample.class(in = 908) (out= 537)(deflated 40%)
```

```
> jarsigner ListProvidersAppletExample.jar AndersonKey
```

```
Enter Passphrase for keystore: teste123
```

```
jar signed.
```

```
Warning:
```

```
The signer's certificate is self-signed.
```

- Verificando a assinatura do ListProvidersAppletExample.jar:

```
>jarsigner -verify -verbose -certs ListProvidersAppletExample.jar
```

```
s      173 Sun Apr 12 22:35:54 BRT 2020 META-INF/MANIFEST.MF
```

```
>>> Signer
```

```
X.509, CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR
```

```
[certificate will expire on 11/07/20 22:05]
```

```
[Invalid certificate chain: PKIX path building failed:
```

```
sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target]
```

```
335 Sun Apr 12 22:35:54 BRT 2020 META-INF/ANDERSON.SF
```

```
1413 Sun Apr 12 22:35:54 BRT 2020 META-INF/ANDERSON.RSA
```

```
0 Sun Apr 12 22:34:56 BRT 2020 META-INF/
```

```
sm     908 Sun Apr 12 22:24:50 BRT 2020 ListProvidersAppletExample.class
```

```
>>> Signer
```

```
X.509, CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR
```

```
[certificate will expire on 11/07/20 22:05]
```

```
[Invalid certificate chain: PKIX path building failed:
```

```
sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target]
```

- Verificando a assinatura do ListProvidersAppletExample.jar:

```
>jarsigner -verify -verbose -certs ListProvidersAppletExample.jar
```

```
.....  
s = signature was verified  
m = entry is listed in manifest  
k = at least one certificate was found in keystore  
i = at least one certificate was found in identity scope
```

```
- Signed by "CN=Anderson da Silva, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR"  
  Digest algorithm: SHA-256  
  Signature algorithm: SHA256withRSA, 2048-bit key
```

jar verified.

Warning:

This jar contains entries whose certificate chain is invalid. Reason: PKIX path building failed:
sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target

This jar contains entries whose signer certificate is self-signed.

This jar contains entries whose signer certificate will expire within six months.

This jar contains signatures that do not include a timestamp. Without a timestamp, users may not be able to validate this jar after any of the signer certificates expire (as early as 2020-07-11).

- Criando o arquivo ListProvidersAppletExample2.html para apresentação do ListProvidersAppletExample.jar:**

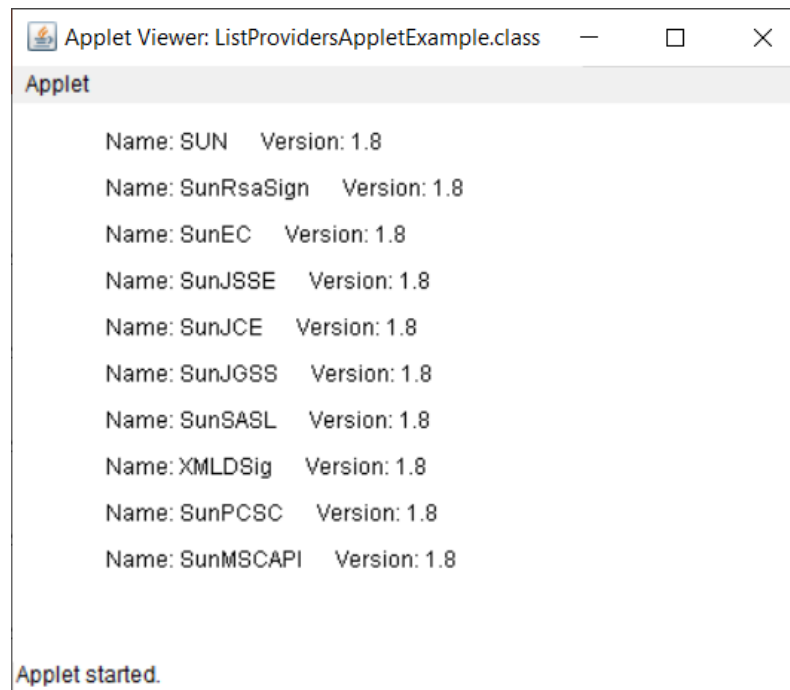
```
<HTML>
<HEAD>
<TITLE> List Providers Applet Example Program </TITLE>
</HEAD>
<BODY>
```

```
<APPLET CODE="ListProvidersAppletExample.class" ARCHIVE="ListProvidersAppletExample.jar" WIDTH=300
HEIGHT=350>
</APPLET>
```

```
</BODY>
</HTML>
```

- Executando o JAR do ListProvidersAppletExample:

> appletviewer ListProvidersAppletExample2.html



Suporte a SSL:

- O JDK 1.8.0 oferece suporte à implementação de serviços de rede com SSL/TLS. Basicamente, deve-se utilizar a *SSLServerSocketFactory*, a *SSLServerSocket* e o método *System.setProperty* para o keystore.
- O *HTTPSServerExample* a seguir utiliza essas classes e métodos:
 - *SSLServerSocketFactory sslf = (SSLServerSocketFactory)SSLServerSocketFactory.getDefault()*: Cria uma instância com configurações default.
 - *SSLServerSocket serverSocket = (SSLServerSocket) sslf.createServerSocket(PORT)*: Cria a instância de um *SSLServerSocket* utilizando a porta *PORT*.
 - *System.setProperty("javax.net.ssl.keyStore", keystore_path)*: fornece o caminho do keystore que contém a chave privada e o certificado digital do servidor.
 - *System.setProperty("javax.net.ssl.keyStorePassword", keystore_password)*: fornece a senha de acesso ao keystore.

Suporte a SSL:

- O JDK 1.8.0 oferece suporte à implementação de serviços de rede com SSL/TLS. Basicamente, deve-se utilizar a *SSLServerSocketFactory*, a *SSLServerSocket* e o método *System.setProperty* para o keystore.
- O *HTTPSServerExample* usa métodos úteis da classe *SSLServerSocket*:
 - .getEnabledCipherSuites()*: retorna um `String []` com a lista de conjuntos de cifras que poderão ser negociadas no handshake.
 - .setEnabledCipherSuites(String [] cipherSuites)*: fornece um `String []` com a lista de conjuntos de cifras que poderão ser negociadas no handshake.
 - .getEnabledProtocols()*: retorna um `String []` com a lista de protocolos que poderão ser negociadas no handshake.
 - .setEnabledProtocols(String [] protocols)*: fornece um `String []` com a lista de conjuntos de cifras que poderão ser negociadas no handshake.

- HTTPSServerExample (Parte1):**

```
import java.io.*;
import java.net.*;
import javax.net.ssl.*;
//
// Servidor HTTPS para ilustrar a utilizacao do SSLServerSocketFactory e SSLServerSocket
public class HTTPSServerExample {
    public static void main(String[] args) throws IOException {
        //
        // verifica args e recebe o caminho e a passphrase do keystore
        if (args.length != 2) {
            System.err.println("Usage: java HTTPSServerExample keystore_path keystore_passphrase");
            System.exit(1);
        }
        String keystore_path = args[0];
        String keystore_passphrase = args[1];
        //
        // Configura as propriedades do keystore
        System.setProperty("javax.net.ssl.keyStore", keystore_path);
        System.setProperty("javax.net.ssl.keyStorePassword", keystore_passphrase);
```

- HTTPSServerExample (Parte2):**

```
//  
// cria um SSL socket usando a factory e alocando a porta 8080  
SSLServerSocketFactory sslsf =  
    (SSLServerSocketFactory)SSLServerSocketFactory.getDefault();  
SSLServerSocket sslss = (SSLServerSocket)sslsf.createServerSocket(8080);  
//  
// loop infinito  
while (true) {  
    try {  
        //  
        // lista as cifras habilitadas para o handshake  
        System.out.println("Listing default cipher suites for handshake...");  
        String[] ciphers = sslss.getEnabledCipherSuites();  
        for (int i = 0; i < ciphers.length; ++i)  
            System.out.println(ciphers[i]);  
    }  
}
```

- HTTPSServerExample (Parte3):**

```
//  
// restringe as cifras para o handshake  
System.out.println("Restricting ciphers available for handshake...");  
ciphers = new String[] {  
    "TLS_AES_128_GCM_SHA256",  
    "TLS_AES_256_GCM_SHA384",  
    "TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384",  
    "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256",  
    "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384" };  
  
sslss.setEnabledCipherSuites(ciphers);  
//  
// lista as cifras habilitadas para o handshake  
System.out.println("Listing cipher suites setting for handshake...");  
ciphers = sslss.setEnabledCipherSuites();  
for (int i = 0; i < ciphers.length; ++i)  
    System.out.println(ciphers[i]);
```

- HTTPSServerExample (Parte4):**

```
//  
// lista os protocolos habilitadas para o handshake  
System.out.println("Listing default protocols for handshake...");  
String[] protocols = ssls.getEnabledProtocols();  
for (int i = 0; i < protocols.length; ++i)  
    System.out.println(protocols[i]);  
  
//  
// restringe os protocolos para TLS1.2 e TLS1.3  
System.out.println("Restricting protocols to TLS v1.2 e v1.3...");  
protocols = new String[] { "TLSv1.2", "TLSv1.3" };  
ssls.setEnabledProtocols(protocols);  
  
//  
// lista os protocolos habilitadas para o handshake  
System.out.println("Listing protocol settings for handshake.....");  
protocols = ssls.getEnabledProtocols();  
for (int i = 0; i < protocols.length; ++i)  
    System.out.println(protocols[i]);
```

- HTTPSServerExample (Parte5):**

```
//  
// bloqueia aguardando a conexao do cliente  
System.out.println( "Accepting requests..." );  
        Socket s = sslss.accept();  
System.out.println( "Client connection made..." );  
// recebe a equisicao do cliente  
System.out.println( "Reading from socket..." );  
BufferedReader in = new BufferedReader(  
    new InputStreamReader(s.getInputStream()));  
System.out.println(in.readLine());  
//  
// cria uma resposta no formato HTML  
        System.out.println( "Sending Hello HTML..." );  
PrintWriter out = new PrintWriter( s.getOutputStream() );  
out.println("HTTP/1.1 200 OK");  
        out.println("Content-Type: text/html; charset=utf-8");  
        out.println("");  
        out.println("<HTML><HEAD><TITLE>HTTPS Server Example</TITLE>" +  
"</HEAD><BODY><H1>Hello World!</H1></BODY></HTML>");
```

- HTTPSServerExample (Parte6):

```
//  
// fecha o stream e o socket  
                System.out.println( "Closing socket..." );  
out.close();  
s.close();  
} catch (Exception e) {  
    e.printStackTrace();  
}  
}  
}  
}
```

- Criando o o par de chaves assimétricas e o certificado para o servidor: (parte 1)**

> keytool -genkey -v -keyalg RSA -alias ServidorKey -keystore sslKeyStore -storetype pkcs12

Enter keystore password: teste123

What is your first and last name?

[Unknown]: localhost

What is the name of your organizational unit?

[Unknown]: DI

What is the name of your organization?

[Unknown]: PUC-Rio

What is the name of your City or Locality?

[Unknown]: Rio de Janeiro

What is the name of your State or Province?

[Unknown]: Rio de Janeiro

What is the two-letter country code for this unit?

[Unknown]: BR

Is CN=localhost, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR correct?

[no]: yes

Generating 2.048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 90 days

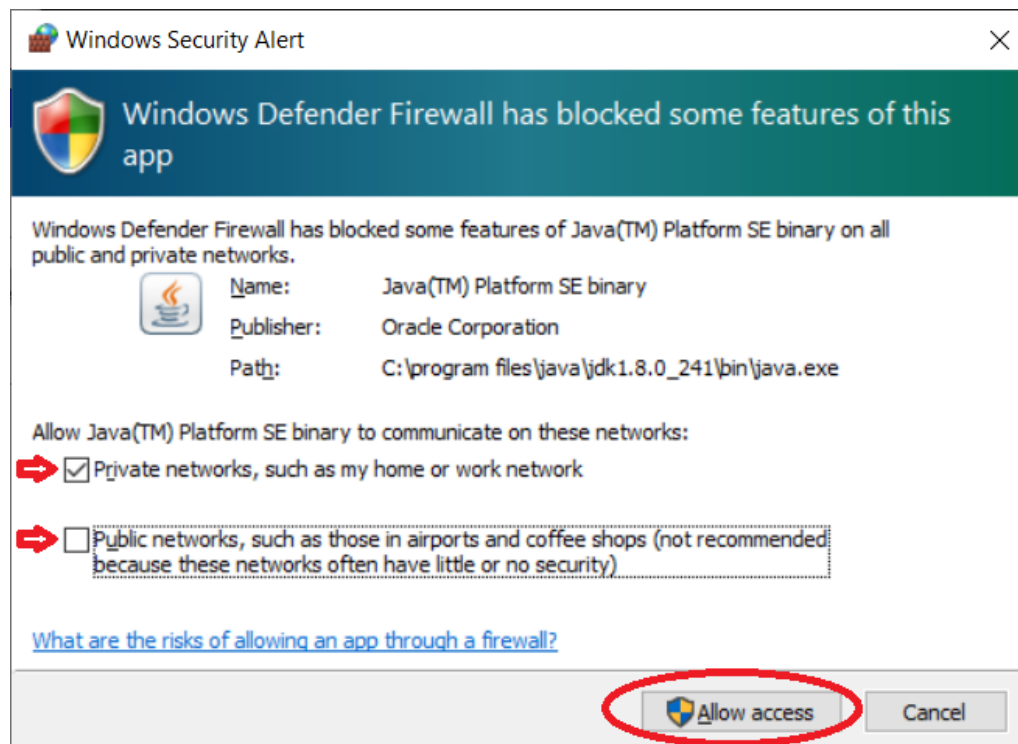
for: CN=localhost, OU=DI, O=PUC-Rio, L=Rio de Janeiro, ST=Rio de Janeiro, C=BR

[Storing sslKeyStore]

- Compilando e executando o HTTPSServerExample:

> javac HTTPSServerExample.java

> java HTTPSServerExample ./sslKeyStore teste123



- Compilando e executando o HTTPServerExample:

```
> javac HTTPServerExample.java
```

```
> java HTTPServerExample ./sslKeyStore teste123
```

Listing default cipher suites for handshake...

TLS_AES_128_GCM_SHA256

TLS_AES_256_GCM_SHA384

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

.....

TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA

TLS_ECDH_RSA_WITH_AES_128_CBC_SHA

TLS_DHE_RSA_WITH_AES_128_CBC_SHA

TLS_DHE_DSS_WITH_AES_128_CBC_SHA

TLS_EMPTY_RENEGOTIATION_INFO_SCSV

- Compilando e executando o HTTPSServerExample:

```
> javac HTTPSServerExample.java
```

```
> java HTTPSServerExample ./sslKeyStore teste123
```

```
Restricting ciphers available for handshake...
```

```
Listing cipher suites setting for handshake...
```

```
TLS_AES_128_GCM_SHA256
```

```
TLS_AES_256_GCM_SHA384
```

```
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
```

```
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
```

```
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
```

- Compilando e executando o HTTPSServerExample:

```
> javac HTTPSServerExample.java
```

```
> java HTTPSServerExample ./sslKeyStore teste123
```

Listing default protocols for handshake...

TLSv1.3

TLSv1.2

TLSv1.1

TLSv1

SSLv2Hello

Restricting protocols to TLS v1.2 e v1.3...

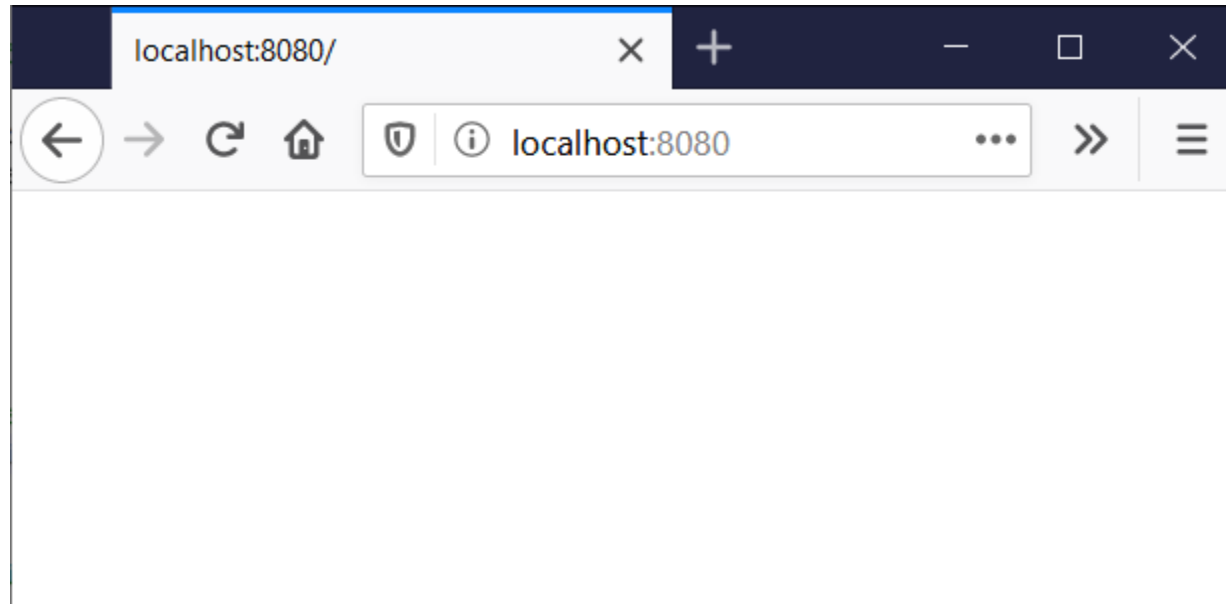
Listing protocol settings for handshake.....

TLSv1.2

TLSv1.3

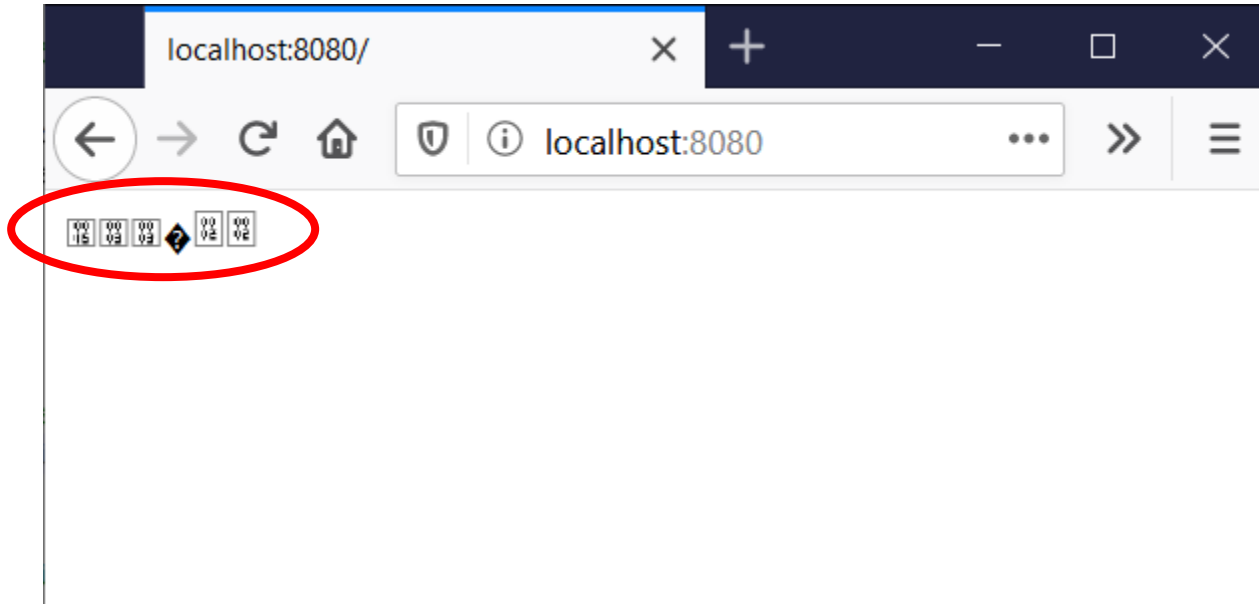
Accepting requests...

- Testando a conexão com o HTTPSServerExample:



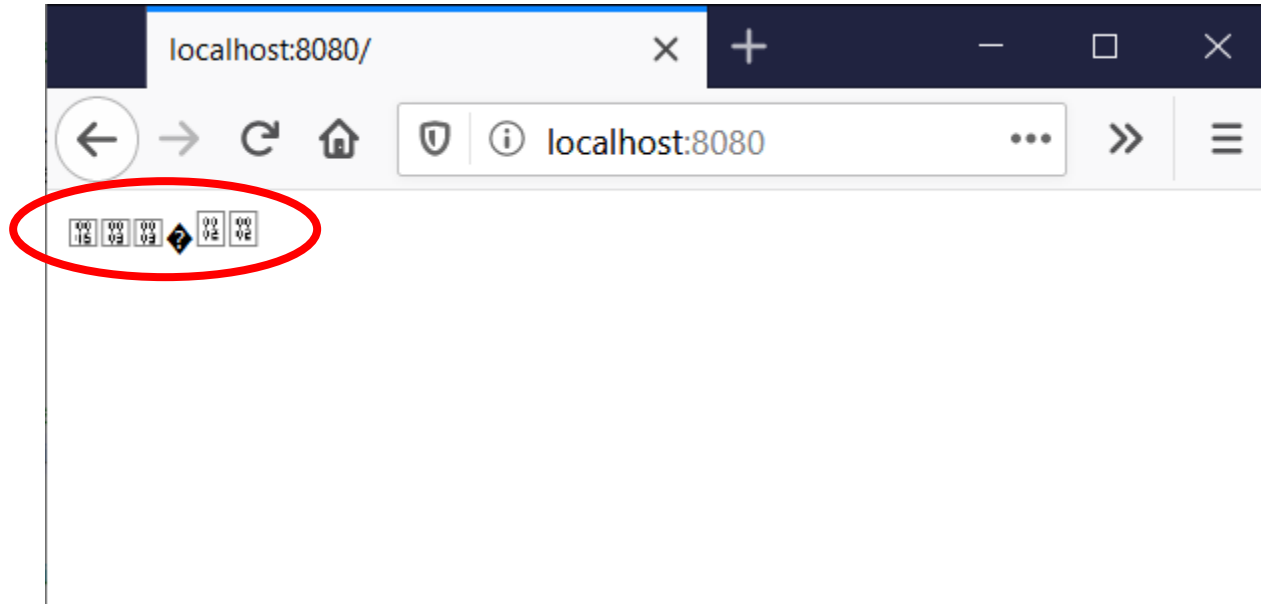
Teste a URL <http://localhost:8080> e verifique o que acontece...

- Testando a conexão com o HTTPServerExample:



Ooops! Por que isso aconteceu?

- Testando a conexão com o HTTPServerExample:



Ooops! Por que isso aconteceu?
A solicitação de conexão não foi HTTPS (HTTP over TLS)!!!

- Testando a conexão com o HTTPSServerExample:

```
> javac HTTPSServerExample.java
```

```
> java HTTPSServerExample ./sslKeyStore teste123
```

```
Accepting requests...
```

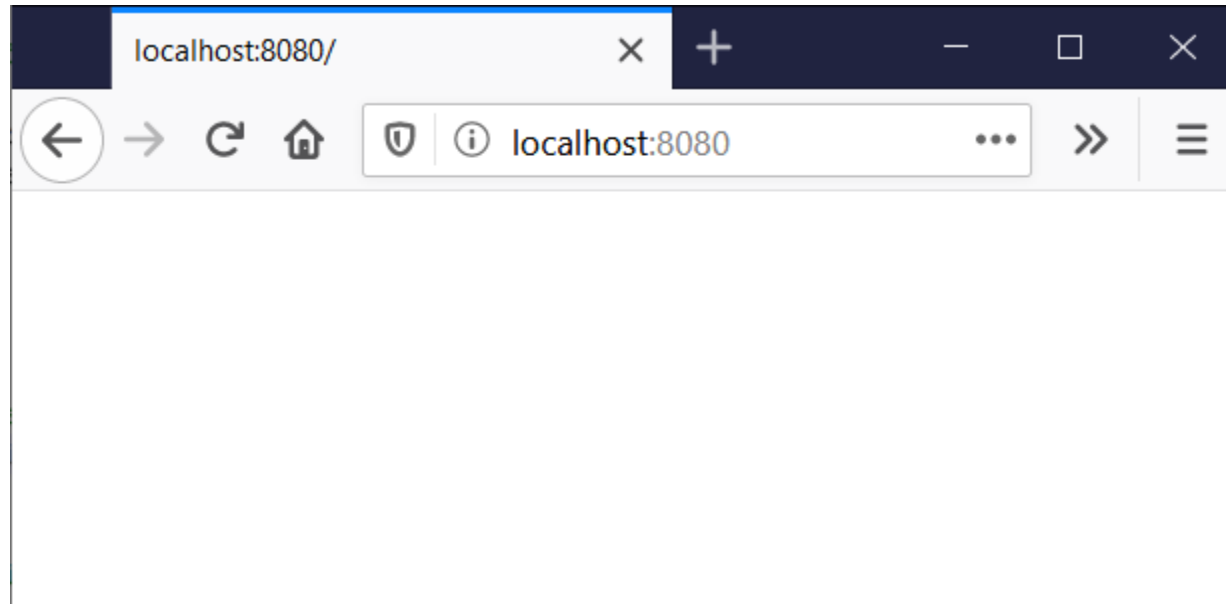
```
Client connection made...
```

```
Reading from socket...
```

```
javax.net.ssl.SSLException: Unsupported or unrecognized SSL message  
    at sun.security.ssl.SSLSocketInputRecord.handleUnknownRecord(SSLSocketInputRecord.java:448)  
    at sun.security.ssl.SSLSocketInputRecord.decode(SSLSocketInputRecord.java:174)  
    at sun.security.ssl.SSLTransport.decode(SSLTransport.java:110)  
    at sun.security.ssl.SSLSocketImpl.decode(SSLSocketImpl.java:1279)  
    at sun.security.ssl.SSLSocketImpl.readHandshakeRecord(SSLSocketImpl.java:1188)  
    at sun.security.ssl.SSLSocketImpl.startHandshake(SSLSocketImpl.java:401)  
    at sun.security.ssl.SSLSocketImpl.ensureNegotiated(SSLSocketImpl.java:808)  
    at sun.security.ssl.SSLSocketImpl.access$200(SSLSocketImpl.java:75)
```

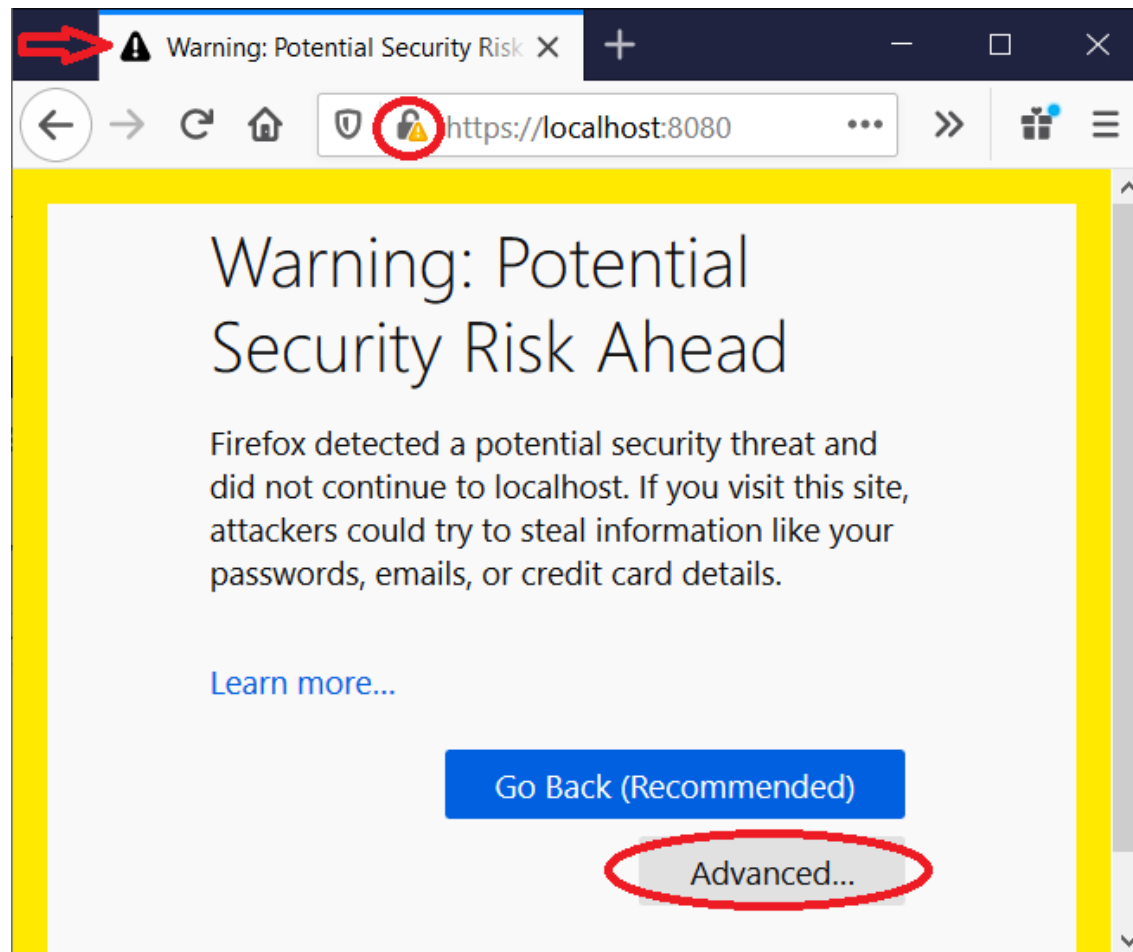
.....

- Testando a conexão com o HTTPServerExample:



Teste a URL <https://localhost:8080> e verifique o que acontece...

- Testando a conexão com o HTTPServerExample:



- Testando a conexão com o HTTPSServerExample:

```
> javac HTTPSServerExample.java
```

```
> java HTTPSServerExample ./sslKeyStore teste123
```

```
Accepting requests...
```

```
Client connection made...
```

```
Reading from socket...
```

```
javax.net.ssl.SSLEnabledException: readHandshakeRecord
```

```
    at sun.security.ssl.SSLSocketImpl.readHandshakeRecord(SSLSocketImpl.java:1199)
```

```
    at sun.security.ssl.SSLSocketImpl.startHandshake(SSLSocketImpl.java:401)
```

```
    at sun.security.ssl.SSLSocketImpl.ensureNegotiated(SSLSocketImpl.java:808)
```

```
    at sun.security.ssl.SSLSocketImpl.access$200(SSLSocketImpl.java:75)
```

```
    at sun.security.ssl.SSLSocketImpl$AppInputSteam.read(SSLSocketImpl.java:891)
```

```
.....  
Suppressed: java.net.SocketException: Software caused connection abort: socket write error
```

```
    at java.net.SocketOutputStream.socketWrite0(Native Method)
```

```
    at java.net.SocketOutputStream.socketWrite(SocketOutputStream.java:111)
```

```
    at java.net.SocketOutputStream.write(SocketOutputStream.java:155)
```

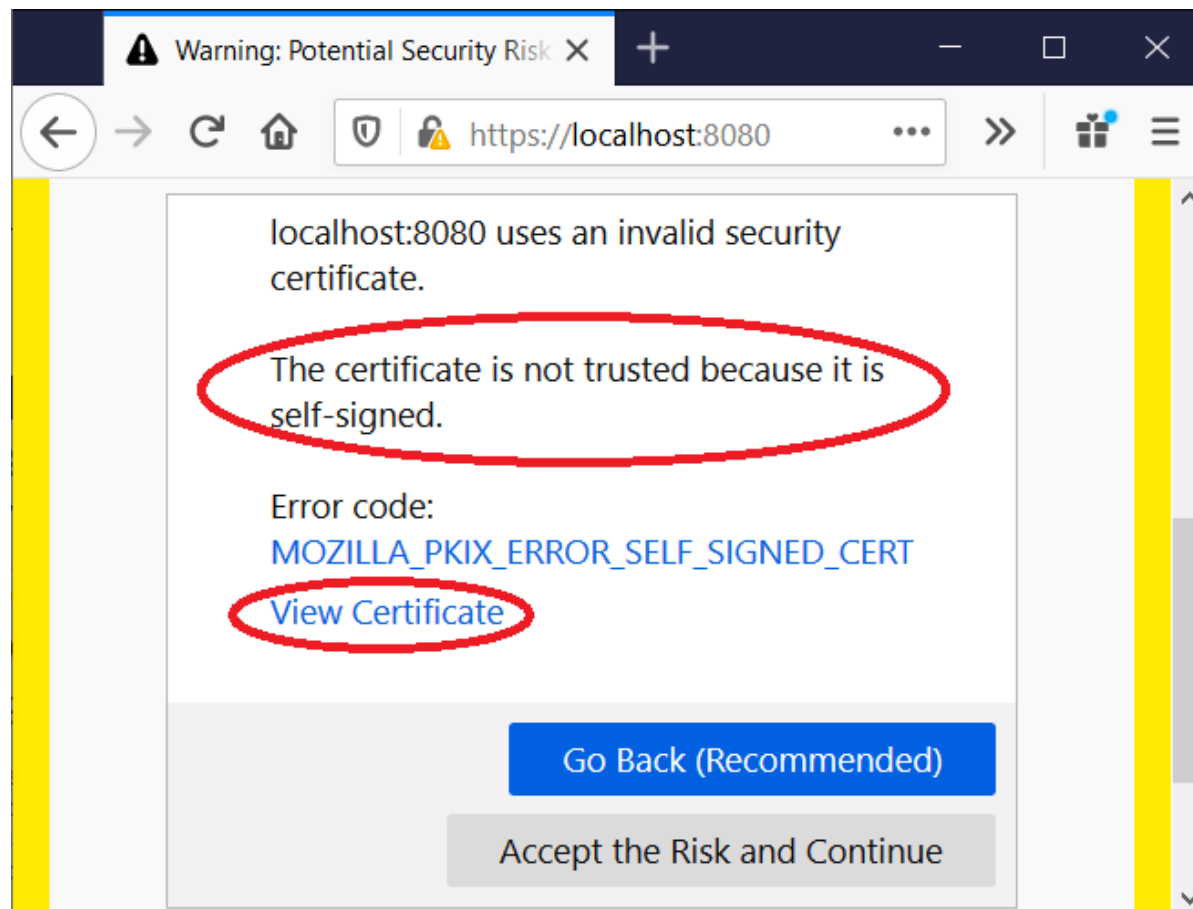
```
    at sun.security.ssl.SSLSocketOutputRecord.encodeAlert(SSLSocketOutputRecord.java:83)
```

```
    at sun.security.ssl.TransportContext.fatal(TransportContext.java:384)
```

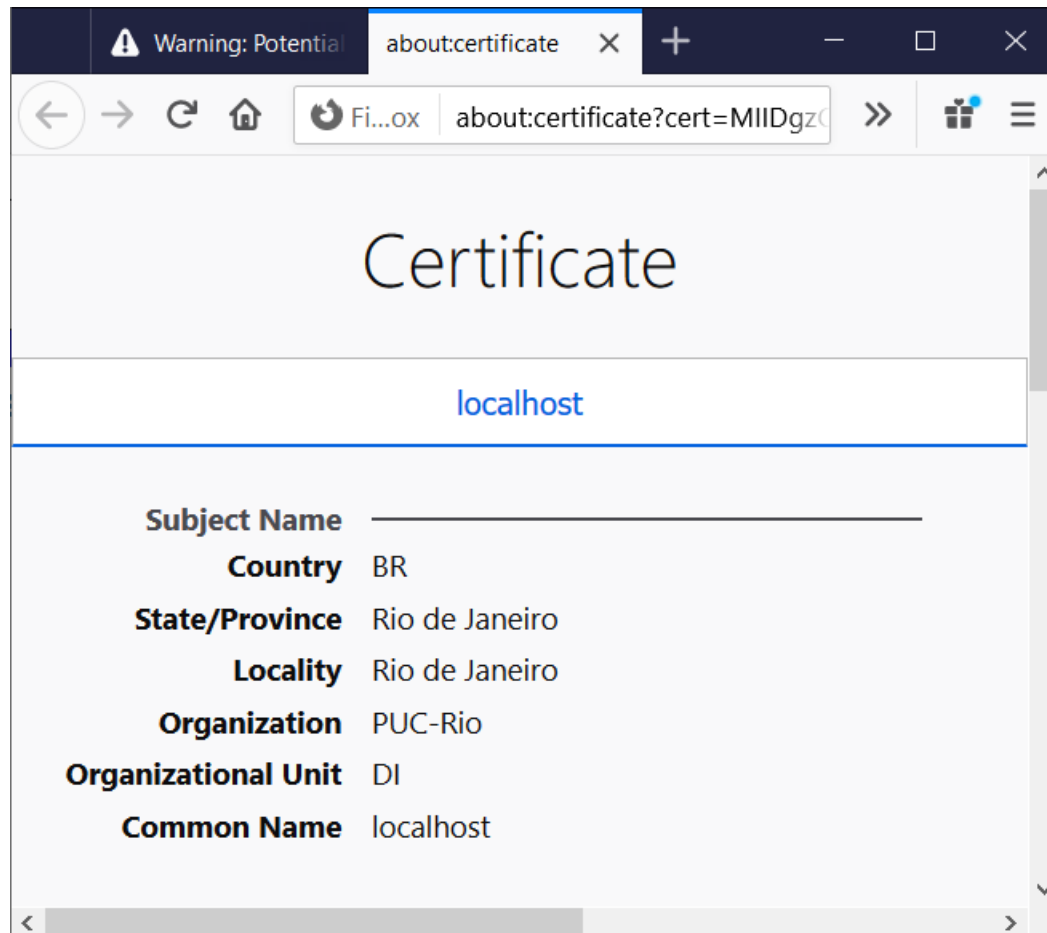
```
    at sun.security.ssl.TransportContext.fatal(TransportContext.java:296)
```

```
    at sun.security.ssl.SSLSocketImpl.startHandshake(SSLSocketImpl.java:411)
```

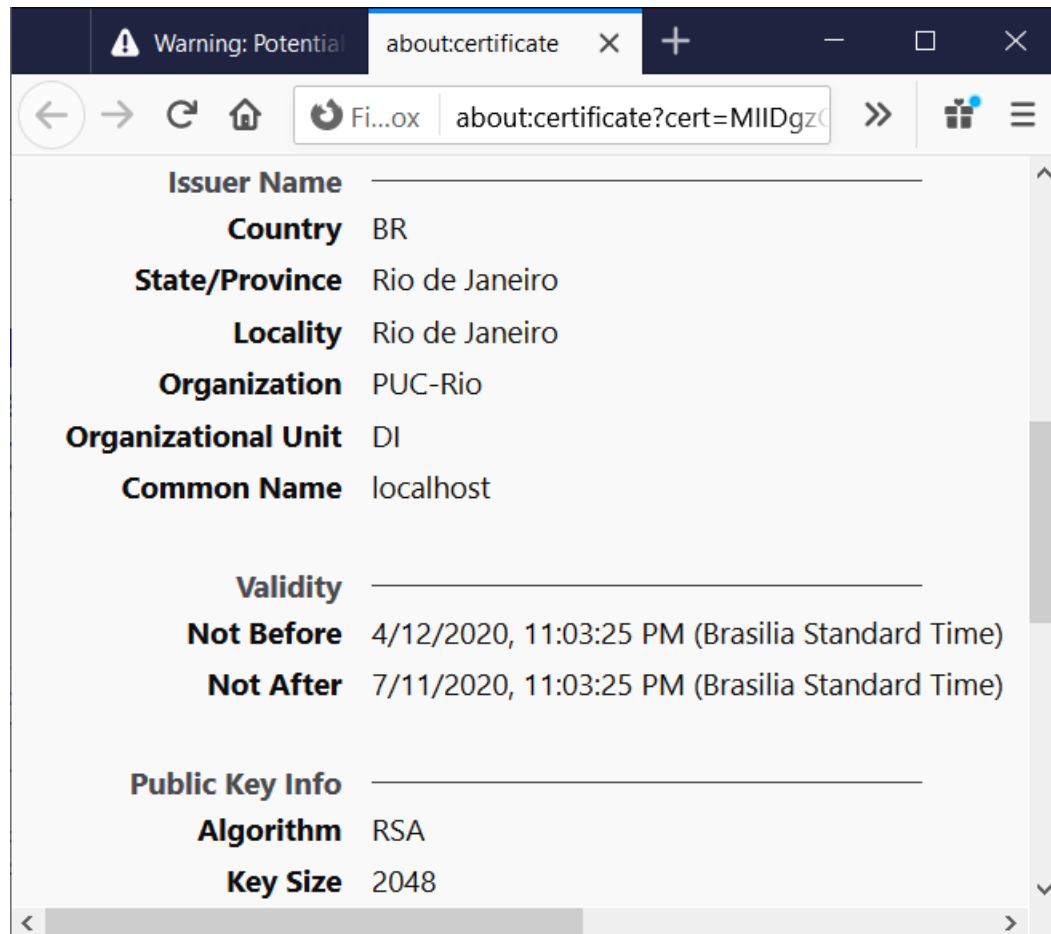
- Testando a conexão com o HTTPServerExample:



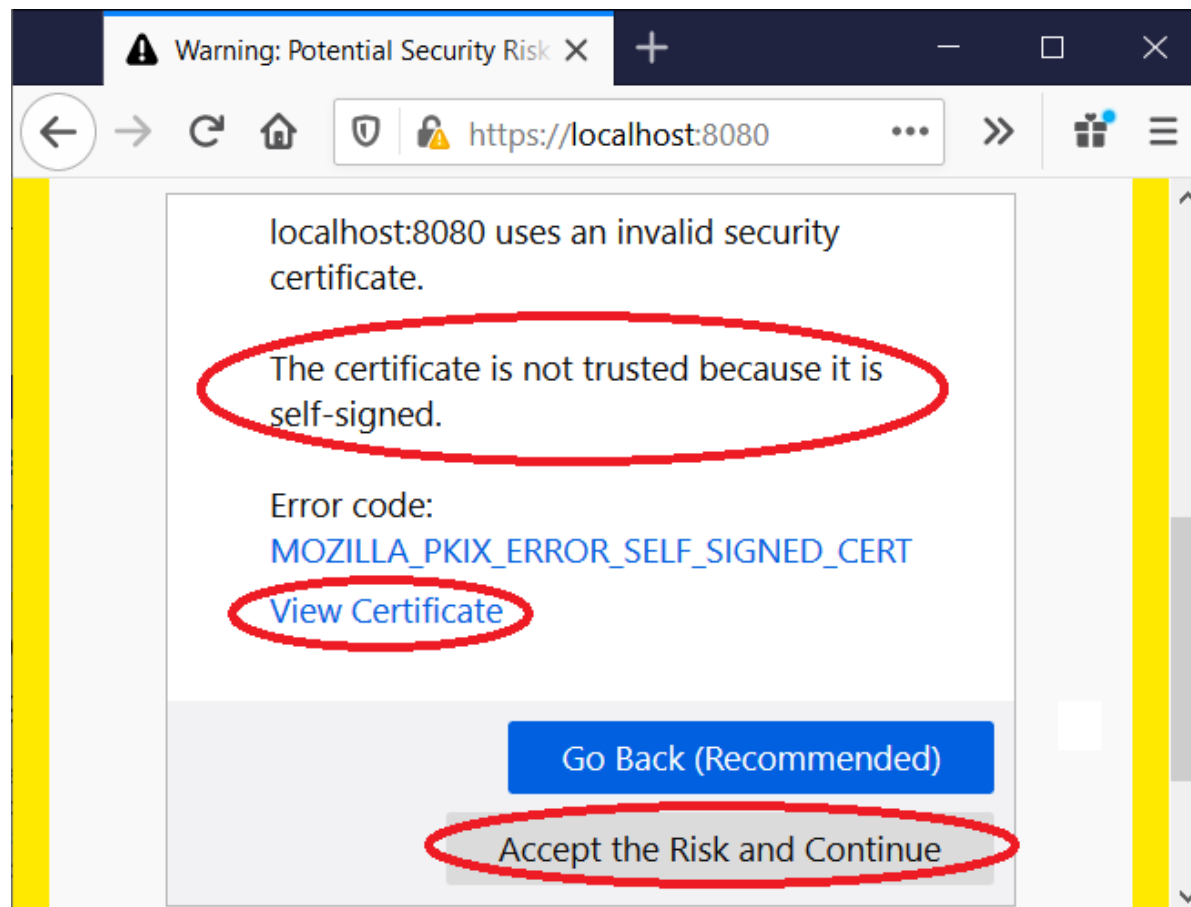
- Testando a conexão com o HTTPServerExample:



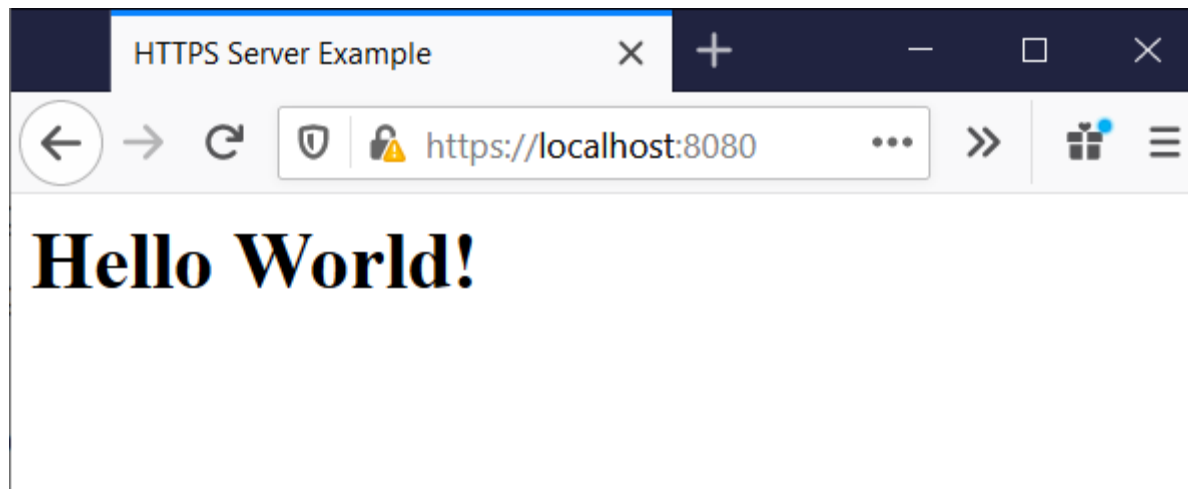
- Testando a conexão com o HTTPServerExample:



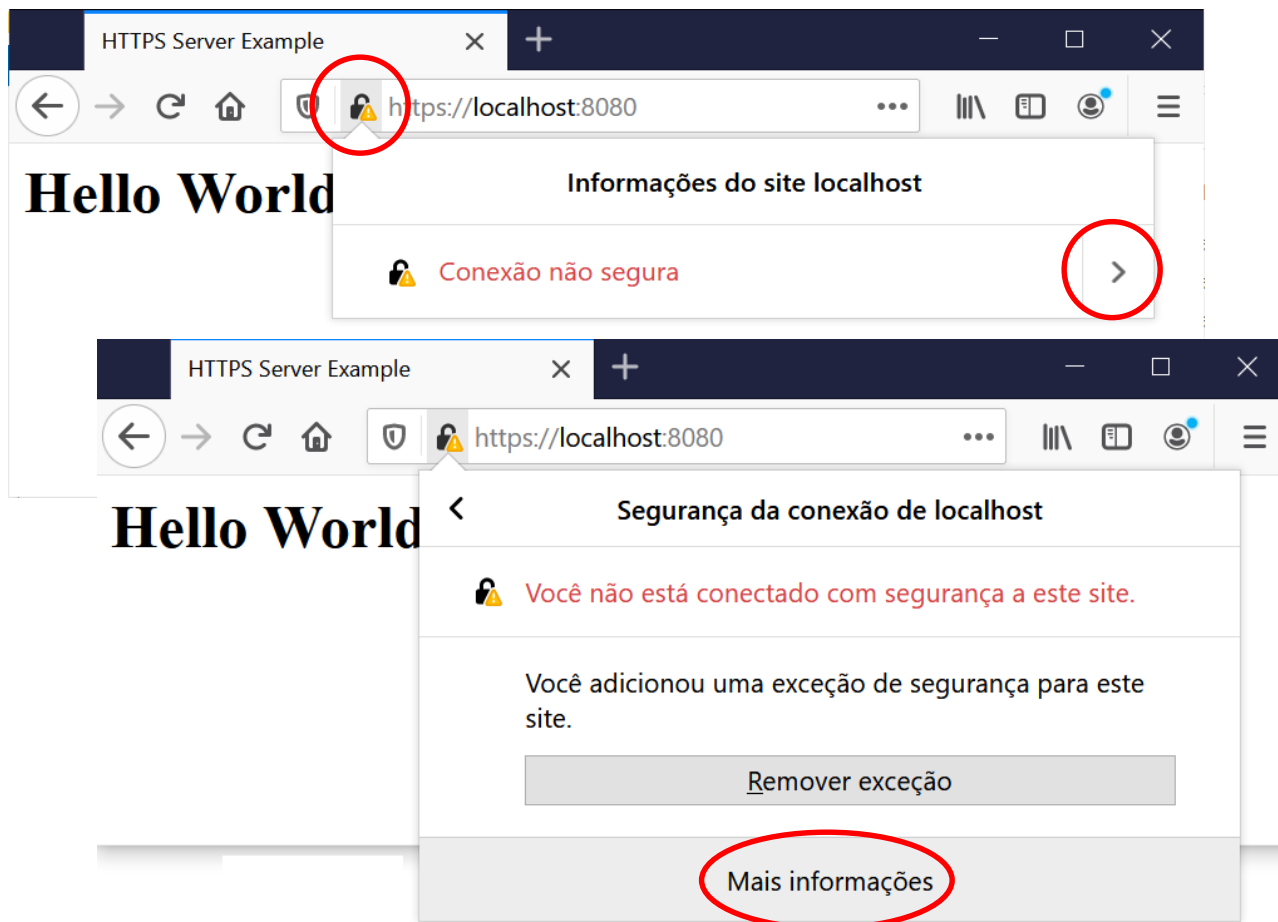
- Testando a conexão com o HTTPServerExample:



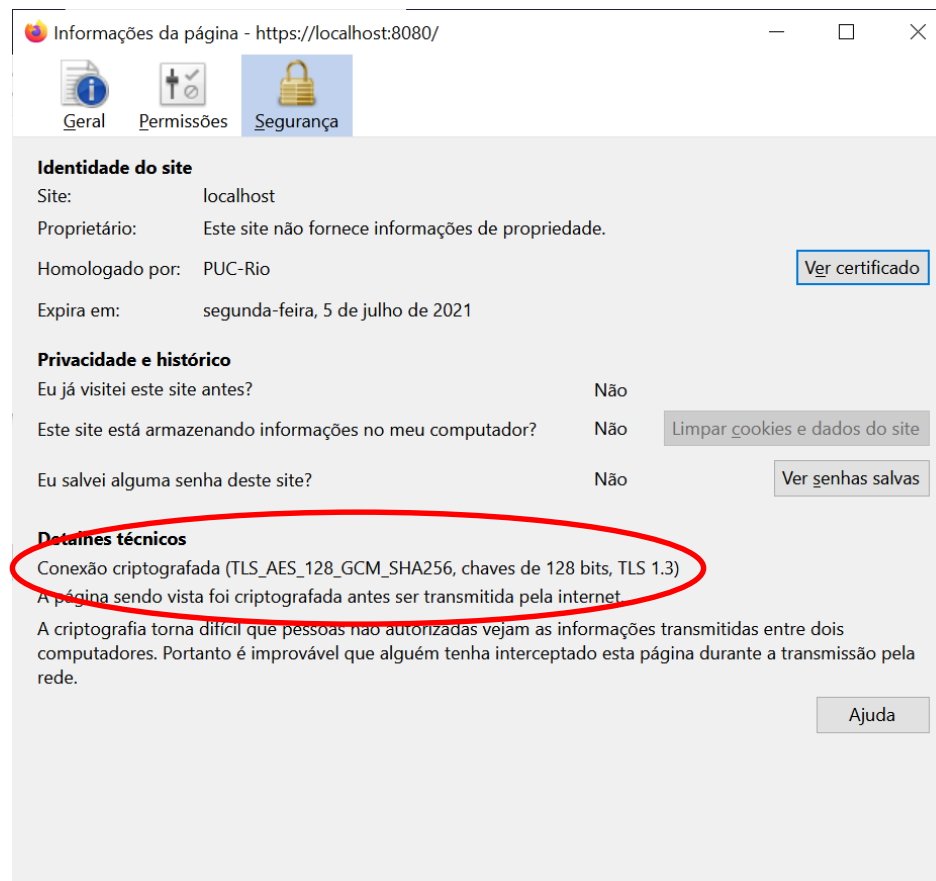
- Testando a conexão com o HTTPSServerExample:



- Testando a conexão com o HTTPSServerExample:



- Testando a conexão com o HTTPServerExample:



- Testando a conexão com o HTTPSServerExample:

```
> javac HTTPSServerExample.java
```

```
> java HTTPSServerExample ./sslKeyStore teste123
```

```
.....  
Accepting requests...
```

```
Client connection made...
```

```
Reading from socket...
```

```
GET / HTTP/1.1
```

```
Sending Hello HTML...
```

```
Closing socket...
```